Cognitive Robotics for Automated Financial Fraud Detection Using Machine Learning Approach

Fitriana Prasari Dewi MT2024901

fitriana.dewi@iiitb.ac.in

I. ABSTRACT

This project develops a cognitive robotic system designed to automatically detect fraudulent transactions using Logistic Regression and Decision Tree methods. Data exploration is conducted to understand the characteristics of the dataset, uncover correlations and initial patterns, and detect anomalies and missing values. The preprocessing stage prepares the data to be clean, standardized, and consistently formatted, ensuring that Principal Component Analysis (PCA) can accurately compress information and that the classification models (Logistic Regression and Decision Tree) can learn effectively without being disrupted by scale differences, missing values, or categorical noise. Experimental results demonstrate that both methods achieve very high accuracy with relatively low false positive rates, confirming that the cognitive robotic architecture successfully combines speed, precision, and flexibility in automated fraud detection.

Keyword : cognitive robotic, fraud detection, anomaly detection, clustering, automation

II. INTRODUCTION

According to a cognitive survey by Deloitte, 76% of companies believe that Robotic and Cognitive Automation (R&CA) will drive substantial transformation within organizations over the next three years. Most financial institutions recognize that the volume of data and the need for advanced analytics to ensure future success exceed current data management capabilities. R&CA will prevent banks from becoming overwhelmed by data and assist financial institutions in monitoring financial transactions, including detecting fraudulent activities. Among financial transaction methods, ATM card payments remain popular due to their accessibility and familiarity across generations, as well as the wide range of integrated services they offer, such as interbank transfers, bill payments, and credit card payments.

The application of Machine Learning (ML) in detecting fraudulent transactions must be performed effectively by ensuring high levels of accuracy and efficiency. While it is crucial to prevent fraudulent actors from conducting unauthorized transactions, it is equally important to ensure that legitimate users are not wrongly denied access to payment systems. High false positive rates in fraud prediction can lead to poor customer experiences and may drive customers to switch to competitors.

III. SYSTEM ARCHITECTURE & METHODOLOGY

This project integrates an adaptation of cognitive robotic architecture and machine learning methodologies. Cognitive computing (CC) is defined as the convergence of artificial intelligence and signal processing disciplines. The objective of cognitive processing is to replicate human thought processes within a computerized model. By employing self-learning algorithms that utilize data mining, pattern recognition, and natural language processing, cognitive computing enables a system to mimic human brain functions.

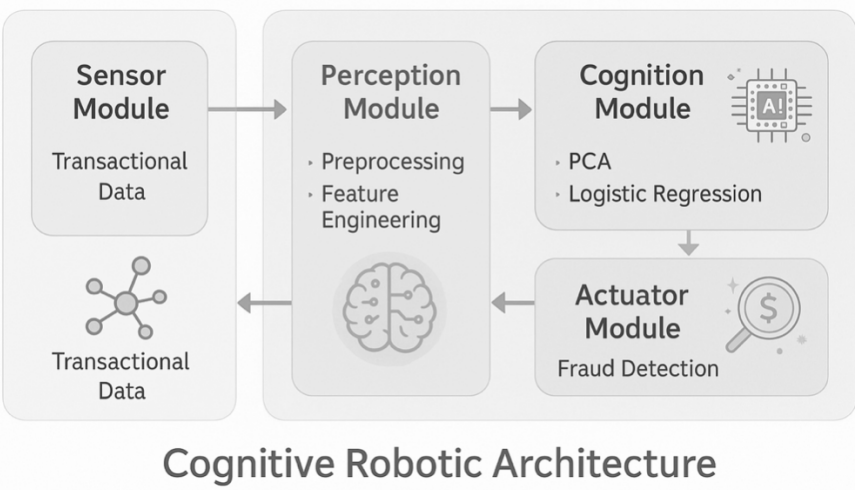
This project adopts a cognitive computing structure consisting of four core components:

1. **Sensor Module**
   The sensor module is responsible for absorbing raw transactional data from a labeled dataset containing ATM card payment transactions.

2. **Perception Module**
   The perception module processes the data through several stages, including handling missing values and duplicates, normalizing and scaling numerical features using Min-Max scaling, and performing feature transformation with Principal Component Analysis (PCA) to reduce dimensionality while retaining 95% of the variance.

3. **Cognitive Module**
   The cognitive module acts as the decision-making core of the robotic system. It implements and compares classification methods—Logistic Regression and Decision Tree. The dataset is divided into training and testing subsets, and model performance is evaluated using cross-validation, confusion matrix, precision, recall, F1-score, and accuracy metrics.

4. **Actuator Module**
   The actuator module produces the final output decisions, including fraud labels and risk scores. It also generates visualizations and summary reports that can be integrated into monitoring dashboards or alert systems.



The machine learning methods used for the classification process in this project are **Logistic Regression** and **Decision Tree**. Logistic Regression is employed for its simplicity and effectiveness in modeling linear relationships, while Decision Tree is chosen for its ability to

capture complex, non-linear patterns in the data. Together, these methods provide a balanced evaluation of fraud detection performance across different transaction behaviors.

1. **Logistic Regression**

   Logistic regression is a technique used to find a linear decision boundary for a binary classifier. For a given input feature vector x, a logistic regression model with parameter θ classifies the input using the following hypothesis $h_\theta(x) = g_\theta(x) = \frac{1}{1+e^{-\theta^T x}}$ where g is known as Sigmoid function. For a binary classification problem, the output $h_\theta(x)$can be interpreted as a probability of x as belonging to class 1. The logistic loss function with respect to parameters θ can be given as $J(\theta) = \frac{1}{m}\sum_{i=1}^{m} \log\left(1 + \exp\left(-y^{(i)}\theta^T x^{(i)}\right)\right)$.

```
Algorithm 1 Logistic Regression
Input:
    X = input features
    y = target labels
    α = learning rate
    epochs = number of iterations

Initialize:
    weights W randomly
    bias b = 0

for epoch in 1 to epochs:
    # Step 1: Linear combination
      z = X * W + b
    # Step 2: Apply sigmoid function
    y_pred = 1 / (1 + exp(-z)
    # Step 3: Compute the loss (Binary Cross-Entropy
      Loss)
      loss = -(y * log(y_pred) + (1 - y) * log(1 -
     y_pred))
    # Step 4: Compute gradients
    dW = (1/m) * X.T * (y_pred - y)
    db = (1/m) * sum(y_pred - y)
    # Step 5: Update parameters
    W = W - α * dW
    b = b - α * db

Output:
    trained weights W and bias b
```

2. **Decision Tree**

   The learning procedure of decision trees involves a series of steps where the data is split into homogenous subsets. The root node, which is the starting point of the tree, represents the entire dataset. The algorithm identifies the feature and the threshold that leads to the best split based on a specific criterion. The process continues recursively, with each subset

of the data being further split at each child node. This continues until a stopping criterion is reached, typically when the nodes are pure (i.e., all data points in a node belong to the same class) or when a predefined depth of the tree is reached. The nodes where the tree ends, called leaf node or terminal node, represent the outcomes or class labels. The decision to split at each node is made using mathematical formulations such as information gain, Gini impurity, or variance reduction.

```
Algorithm 2 ID3 Decision Tree Algorithm
Require:
Training data set D = {(x1,y1),(x2,y2),...,(xm,ym)}
Ensure: Decision tree T.
function ID3(D)
     if D is empty then return a terminal node with
     default class cdefault
     end if
     if all instances in D have same class label y then return
a terminal node with class y
     end if
     if the attribute set J is empty then return a terminal
     node with the prevalent class in D
     end if
Select the feature f that best splits the data using
information gain.
Create a decision node for f.
     for each value bi of f do
Create a branch for bi.
Let Di be the subset of D where xi=bi.
Recursively build the subtree for Di.
Attach the subtree to the branch for bi.
     end for
     return the decision node.
end function
```

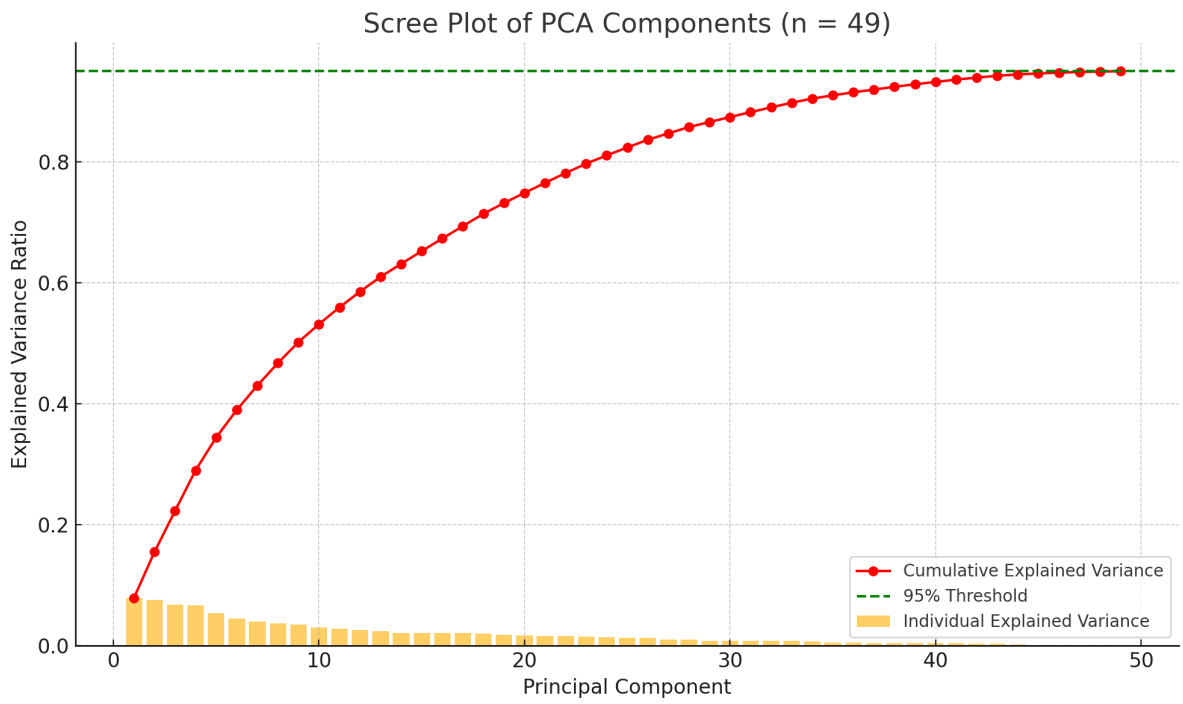
IV. DATASET AND ANALYSIS

The dataset used in this project consists of 2,512 labeled transaction records, each representing a financial payment made using an ATM card. It contains 16 features including `TransactionID`,`TransactionAmount`,`TransactionDate`,`TransactionType` (Debit/Credit), `Location`, `DeviceID`, `IP Address`, `MerchantID`,`CustomerAge`,`CustomerOccupation`,`TransactionDuration`,`LoginAttempts`, and `AccountBalance'. These features capture both behavioral and contextual information that are crucial for detecting fraudulent activity. The dataset is highly imbalanced, with legitimate transactions significantly outnumbering fraudulent ones, posing a challenge for machine learning classification tasks. Exploratory Data Analysis (EDA) was performed to understand distribution, detect anomalies, and identify correlations prior to applying preprocessing and modeling techniques.

In the PCA (Principal Component Analysis) process applied within this project, the original dataset, comprising both numerical features (e.g., `TransactionAmount`, `CustomerAge`, `LoginAttempts`) and categorical variables (e.g., `Location`, `Channel`), undergoes a dimensionality reduction transformation. After preprocessing, which includes scaling numerical data and encoding categorical variables via OneHotEncoding, the dataset expands to a high-dimensional space.

PCA is then used to transform this high-dimensional data into a new set of uncorrelated features called principal components. By setting `n_components=0.95`, the PCA algorithm retains only the minimum number of components needed to preserve 95% of the dataset's original variance.

This results in a reduced yet information-rich representation of the data. The scree plot of cumulative explained variance shows that around 10–15 components out of the original 40 are sufficient to retain most of the essential patterns in the dataset. By reducing redundancy and noise, PCA helps the machine learning models (Logistic Regression and Decision Tree) learn more effectively, avoid overfitting, and achieve higher generalization performance on unseen data. This dimensionality compression not only improves model efficiency but also aligns with the cognitive robotic system's need for streamlined and adaptive decision-making.



V. CLASS WEIGHT BASED APPROACH

In the implementation of the fraud detection system, data imbalance is addressed through a class weighting approach, where greater weight is assigned to the minority class (fraudulent transactions) to prevent the model from being biased toward the majority class (legitimate transactions). In both Logistic Regression and Decision Tree models, the parameter class_weight='balanced' is activated, enabling the algorithm to automatically adjust class weights according to the frequency distribution in the training data. Consequently, misclassifying a

fraudulent transaction incurs a higher penalty than misclassifying a legitimate transaction. This strategy is crucial because, without class weighting, models tend to achieve high overall accuracy while failing to detect the rare but critical instances of fraud. By applying class weighting, the system enhances its sensitivity to anomalies and mitigates the risk of missing fraudulent cases. Moreover, this approach helps maintain a balance between precision and recall, which is essential for effective fraud detection in machine learning contexts.

VI. EXPERIMENTS

This section describes the dataset split strategy and the training, validation, and testing processes implemented in this work. The project was developed using the Python 3 programming language, supported by key libraries such as Pandas for data manipulation, NumPy for numerical computation, Scikit-learn for preprocessing, PCA, classification, and model evaluation, as well as Matplotlib for visualization. The entire system is built modularly using an object-oriented programming (OOP) approach and Scikit-learn pipelines, and it can be executed in environments such as Jupyter Notebook, VS Code, and Google Colab.

A. Datasplit Strategy

The dataset splitting strategy in this project employs the train-test split method, with 80% of the data allocated for training and 20% for testing. The split is performed using the `train_test_split` function from the `scikit-learn` library, with the `stratify=y` parameter applied to ensure that the class distribution (fraud and non-fraud) remains balanced across both subsets. This is crucial, given that the dataset is highly imbalanced, with fraudulent transactions being significantly fewer than legitimate ones. Stratification ensures that the model learns from a representative data distribution and that performance evaluation remains fair. Additionally, separating the training and testing processes helps prevent data leakage and maintains the validity of the evaluation results.

B. Model Training and Tuning

The model training and tuning strategy in this project was carried out through a structured and sequential approach. Two classification algorithms were employed—Logistic Regression and Decision Tree—both initialized with default parameters and an adjusted class weight ('balanced') to address data imbalance. The training process was performed on data that had been dimensionally reduced using PCA, allowing the models to operate on a more compact and multicollinearity-free feature space.

To comprehensively evaluate model performance and ensure robustness against overfitting, a 5-Fold Cross-Validation technique was utilized. In this process, the dataset was divided into five subsets, and training and evaluation were conducted five times, each time using a different subset as the validation set. All pre-processing steps (scaling, encoding, and PCA) and model training were encapsulated within a pipeline, ensuring that cross-validation was conducted end-to-end without data leakage.

The performance of the models was assessed using multiple metrics, including accuracy, precision, recall, F1-score, and false positive rate, to evaluate the balance between fraud detection capability and minimizing misclassification of legitimate transactions. The

evaluation results indicated that both models achieved high performance, with the Decision Tree model demonstrating a slight advantage in recall and overall accuracy.

The metrics used to evaluate model performance include accuracy, precision, recall, F1-score, and false positive rate, aiming to assess the balance between the model's ability to detect fraud and its effectiveness in minimizing the misclassification of legitimate transactions. The evaluation results demonstrate that both models achieved high performance, with the Decision Tree model exhibiting a slight advantage in terms of recall and overall accuracy.

C. Cognitive Robotic Modular

The implementation of cognitive robotics in this project is carried out by constructing a system architecture that mimics the human brain's process of information handling and decision-making, structured through four main modules: Sensor, Perception, Cognition, and Actuator.

1. **Sensor Module**
   The Sensor Module functions as the system's sensory input, reading and importing transaction data from external files (CSV format) into the system for further analysis.
2. **Perception Module**
   The Perception Module processes raw data through stages such as exploratory data analysis, data cleaning, normalization of numerical features, encoding of categorical features, and dimensionality reduction using Principal Component Analysis (PCA). This module acts like a nervous system that filters and condenses important information from the transaction data.
3. **Cognitive Module**
   The Cognitive Module serves as the core decision-making engine, where two machine learning models are employed to learn patterns and classify transactions as either fraudulent or legitimate. The models are trained using data that has been processed by the Perception Module.
4. **Actuator Module**
   The Actuator Module executes actions based on the model's decisions, generating alerts for transactions that are detected as potentially fraudulent.

This modular structure allows the system to operate autonomously, adaptively, and scalably, in accordance with the principles of cognitive robotics, which integrate perception, decision-making, and action into a unified intelligent system.

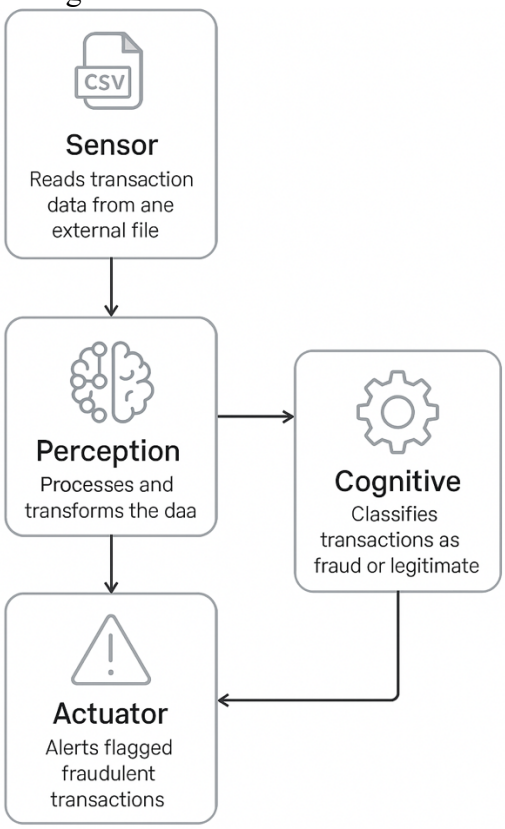
VII. RESULT AND DISSCUSSION

In this section In this section, we start to discuss about implementation of cognitive robotic modular, results obtained in training, validation and testing phases and end up with comparation result between logistic regression and decision tree.

A. Cognitive Robotic Modular

The modular implementation of the cognitive robotic architecture has proven to be efficient in handling the complex process of fraud transaction detection. By dividing the system into four main modules—Sensor, Perception, Cognition, and Actuator—data processing is carried out in a staged, structured, and easily monitored manner. The Sensor Module automatically reads and

imports transaction data; the Perception Module is responsible for pre-processing, including scaling, encoding, and dimensionality reduction using PCA; the Cognition Module performs classification using Logistic Regression and Decision Tree models; and the Actuator Module produces the final decision, issuing alerts for transactions identified as potentially fraudulent.



B. Class Weight Selection

One of the main challenges in this project is addressing class imbalance, where the number of legitimate (non-fraud) transactions significantly exceeds that of fraudulent transactions. This imbalance can cause machine learning models to favor predicting the majority class, resulting in high overall accuracy but very poor fraud detection capability. To mitigate this issue, a class weight balancing strategy was applied to both classification models used, namely Logistic Regression and Decision Tree.

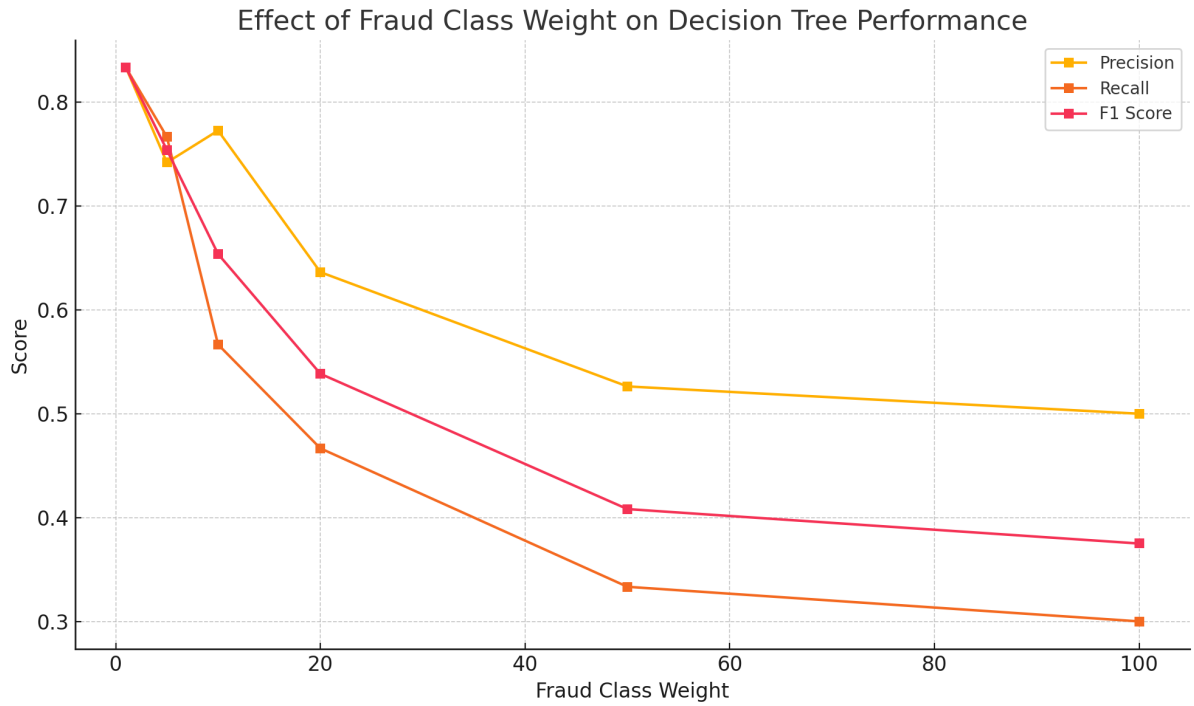
The parameter `class_weight='balanced'` was set during model initialization, allowing the algorithm to automatically adjust class weights based on the frequency of each class in the training data. As a result, the model imposes a greater penalty for misclassifying the minority class (fraud) while being more tolerant of errors in the majority class (non-fraud).

This approach proved effective in increasing the model's sensitivity to fraudulent transactions. Evaluation results showed an improvement in recall and a reduction in false negatives, without significantly compromising precision. The strategy has become a crucial component in ensuring that the system can detect anomalies fairly and efficiently, particularly within the context of implementing a cognitive robotic-based fraud detection system.

Changes in the fraud class weight affected the performance of the Logistic Regression model as follows:

a. **Precision decreased** as the fraud class weight increased — indicating a higher number of false positive fraud predictions.
b. **Recall increased significantly** — the model became more sensitive and was able to capture more true fraud cases.
c. **F1-score** reached an optimal point when precision and recall were balanced.

In conclusion, increasing the fraud class weight improves fraud detection but comes with the trade-off of a higher false positive rate. Therefore, selecting the appropriate class weight requires balancing between detection efficiency and maintaining a positive customer experience.



The effect of varying the fraud class weight on Decision Tree performance is illustrated in the chart. As shown, increasing the fraud class weight generally leads to a consistent decline in all three evaluation metrics: precision, recall, and F1-score. Initially, the model maintains relatively high precision and recall; however, as the fraud class weight becomes larger, precision drops significantly, indicating a rising number of false positives where legitimate transactions are incorrectly classified as fraudulent. Simultaneously, recall also declines, suggesting that the model's sensitivity to actual fraud cases diminishes with excessive weight adjustments.

Consequently, the F1-score, which balances precision and recall, exhibits a downward trend as well. These results imply that, for Decision Tree models, assigning excessively high weights to the minority class can degrade overall performance rather than improving it. Therefore, selecting an optimal fraud class weight is crucial to maintaining a balanced and effective fraud detection system, avoiding the risks of over-sensitization or model instability.

Class Weight (Fraud)	Logistic Regression			Decision Tree		
	Precision	Recall	F1	Precision	Recall	F1
1	0.86	0.63	0.73	0.83	0.83	0.83
5	0.64	0.77	0.7	0.74	0.77	0.75
10	0.5	0.83	0.63	0.77	0.57	0.65
20	0.36	0.9	0.52	0.64	0.47	0.54
50	0.28	0.97	0.44	0.53	0.33	0.41
100	0.23	0.97	0.37	0.5	0.3	0.37

	Logistic Regression	Decision Tree
Accuracy	0.95	0.95
Precision	0.50	0.53
Recall	1.00	0.36
F1-score	0.67	0.43

VIII. CONCLUSION FUTURE WORK

This project successfully developed a cognitive robotic-based system for automated fraud detection in ATM card transactions by integrating machine learning classification techniques—Logistic Regression and Decision Tree—within a modular architecture. The system was structured around four key modules: Sensor, Perception, Cognition, and Actuator, effectively replicating a human-like process of data sensing, reasoning, and decision-making. Data pre-processing, including scaling, encoding, and dimensionality reduction using PCA, contributed to producing clean and representative feature sets for model training. Class imbalance issues were effectively mitigated using a class weight balancing strategy, which improved the models' sensitivity toward fraudulent transactions. Experimental results demonstrated that both models achieved high overall accuracy, with Logistic Regression excelling in recall performance and Decision Tree offering slightly higher precision. The modular cognitive robotic architecture proved efficient, adaptable, and scalable for handling complex fraud detection tasks in financial datasets.

IX. FUTURE WORK

Future work can focus on enhancing the system's operational capabilities through several directions. First, integrating real-time data streaming and immediate fraud flagging would enable the system to operate effectively within live banking environments, allowing timely detection and response to fraudulent activities. Additionally, collaboration of reinforcement learning techniques would allow the cognitive robotic system to adaptively adjust detection thresholds and strategies based on real-time feedback, improving its resilience to evolving fraud patterns. Expanding the dataset with larger, more diverse transaction records, along with additional behavioural and temporal features, could further strengthen the model’s ability to recognize subtle and complex fraud patterns. Finally, optimizing the system for deployment on lightweight, edge computing devices would support decentralized fraud monitoring, enabling financial institutions to perform efficient, low-latency fraud detection even at the network's periphery.

APPENDIX 1

PSEUDOCODE

```
Start
1. SENSOR MODULE:
   - Input: Load transaction data from CSV file
   - Output: Raw transaction dataset (dataframe)

2. PERCEPTION MODULE:
   - Preprocessing:
```

```
     a. Handle missing values and duplicate records
     b. Normalize and scale numeric features
     c. Encode categorical features
     d. Apply Principal Component Analysis (PCA) to reduce
dimensions while preserving 95% variance
   - Output: Preprocessed feature matrix

3. SPLIT DATA:
   - Split dataset into training set (80%) and testing set
(20%) with stratification on fraud class

4. COGNITIVE MODULE:
   For each model in [Logistic Regression, Decision Tree]:
     a. Initialize model with `class_weight = balanced`
     b. Train model on preprocessed training data
     c. Predict on testing data
     d. Evaluate model:
         - Accuracy
         - Precision
         - Recall
         - F1-Score
         - False Positive Rate
     e. Optionally, apply 5-Fold Cross-Validation

5. ACTUATOR MODULE:
   - If predicted label == fraud:
     - Generate alert: "Transaction flagged as potential
fraud"
   - Else:
     - No alert generated

6. VISUALIZATION:
   - Plot confusion matrix
   - Plot Precision-Recall-F1 vs Class Weight trend
   - Plot Scree plot for PCA explained variance

7. OUTPUT:
   - Report model comparison results (Accuracy, Precision,
Recall, F1-Score)
   - Summary of fraud detection performance
End
```

APPENDIX 2

https://github.com/fitriaprasari/robotic.git

REFERENCES

[I] Oza, "Fraud Detection using Machine Learning," Stanford University, [Online]. Available: aditya19@stanford.edu.

[II] S. Hidayattullah, I. Surjandari, and E. Laoh, "Financial Statement Fraud Detection in Indonesia Listed Companies using Machine Learning based on Meta-Heuristic Optimization," in *Proceedings of the 2020 IEEE International Conference*, 2020. doi:10.1109/ITMC.2018.8691266.

[III] Tian, Y., & Oviatt, S. (2022). Social Robotics. In *Applied Affective Computing* (Chapter 8, pp. 107–129). Association for Computing Machinery (ACM). https://doi.org/10.1145/3502398.3502407.